

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8

дисциплина: Архитектура компьютера

Студент: Ксения Танчила

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

Цель работы.....	3
Задание.....	4
Теоритическое введение.....	5
Выполнение лабораторной работы.....	6
Реализация циклов в NASM.....	6
Обработка аргументов командной строки.....	8
Задания для самостоятельной работы.....	12
Выводы.....	13
Список литературы.....	14

1 Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

2 Задание

1. Реализация циклом в NASM
2. Обработка аргументов командной строки
3. Самостоятельное написание программы по материалам лабораторной работы

3 Теоретическое введение

Стек — это структура данных, организованная по принципу LIFO («Last In — First Out» или «последним пришёл — первым ушёл»). Стек является частью архитектуры процессора и реализован на аппаратном уровне. Для работы со стеком в процессоре есть специальные регистры (ss, bp, sp) и команды. Основной функцией стека является функция сохранения адресов возврата и передачи аргументов при вызове процедур. Кроме того, в нём выделяется память для локальных переменных и могут временно храниться значения регистров.

4 Выполнение лабораторной работы

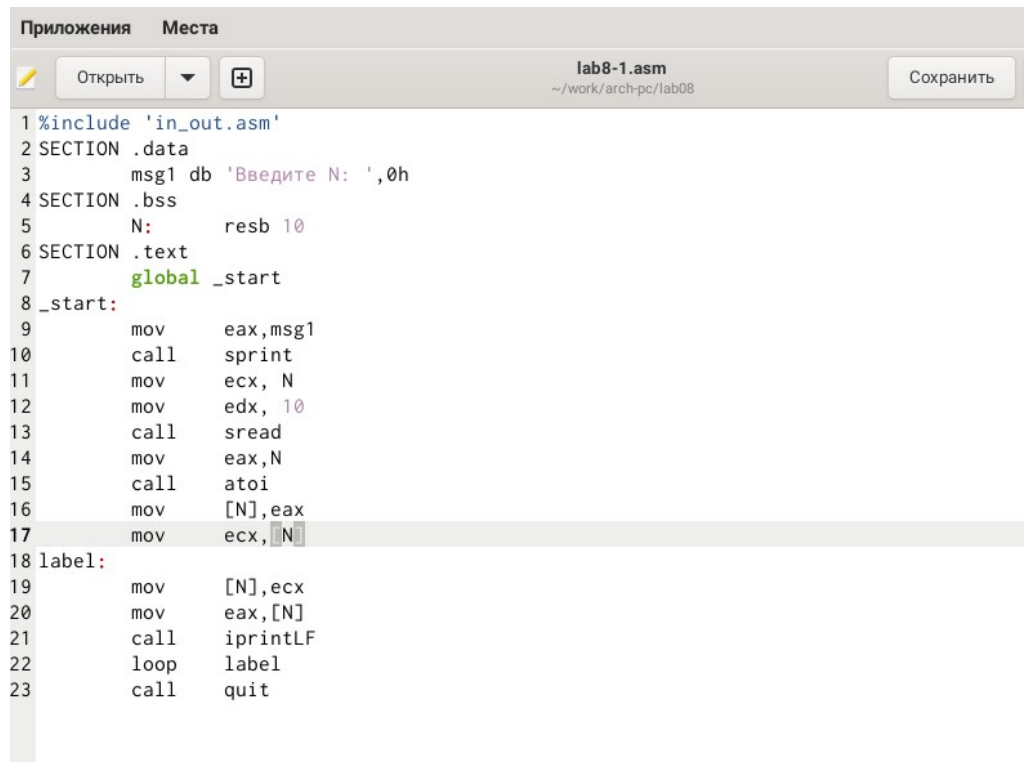
4.1 Реализация циклов в NASM

1. Создала каталог для программам лабораторной работы № 8, перешла в него и создала файл lab8-1.asm.

```
kstanchila@dk3n55 ~ $ mkdir ~/work/arch-pc/lab08
kstanchila@dk3n55 ~ $ cd
Display all 106 possibilities? (y or n)
kstanchila@dk3n55 ~ $ cd
kstanchila@dk3n55 ~ $ cd ~/work/arch-pc/lab08
kstanchila@dk3n55 ~/work/arch-pc/lab08 $ touch lab8-1.asm
kstanchila@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-1.asm
```

Рис. 4.1.1 — Создание каталога

При реализации циклов в NASM с использованием инструкции loop необходимо помнить о том, что эта инструкция использует регистр ecx в качестве счетчика и на каждом шаге уменьшает его значение на единицу. В качестве примера рассмотрели программу, которая выводит значение регистра ecx. Внимательно изучила текст программы Листинга 8.1 и ввела его в файл lab8-1.asm (Рис. 4.1.2). Создала исполняемый файл и проверила его работу (Рис. 4.1.3).



```
Приложения Места
lab8-1.asm
~/work/arch-pc/lab08
Сохранить

1 %include 'in_out.asm'
2 SECTION .data
3     msg1 db 'Введите N: ',0h
4 SECTION .bss
5     N:    resb 10
6 SECTION .text
7     global _start
8 _start:
9     mov     eax,msg1
10    call    sprint
11    mov     ecx, N
12    mov     edx, 10
13    call    sread
14    mov     eax,N
15    call    atoi
16    mov     [N],eax
17    mov     ecx,[N]
18 label:
19    mov     [N],ecx
20    mov     eax,[N]
21    call    iprintLF
22    loop    label
23    call    quit
```

Рис. 4.1.2 — Текст программы lab8-1.asm

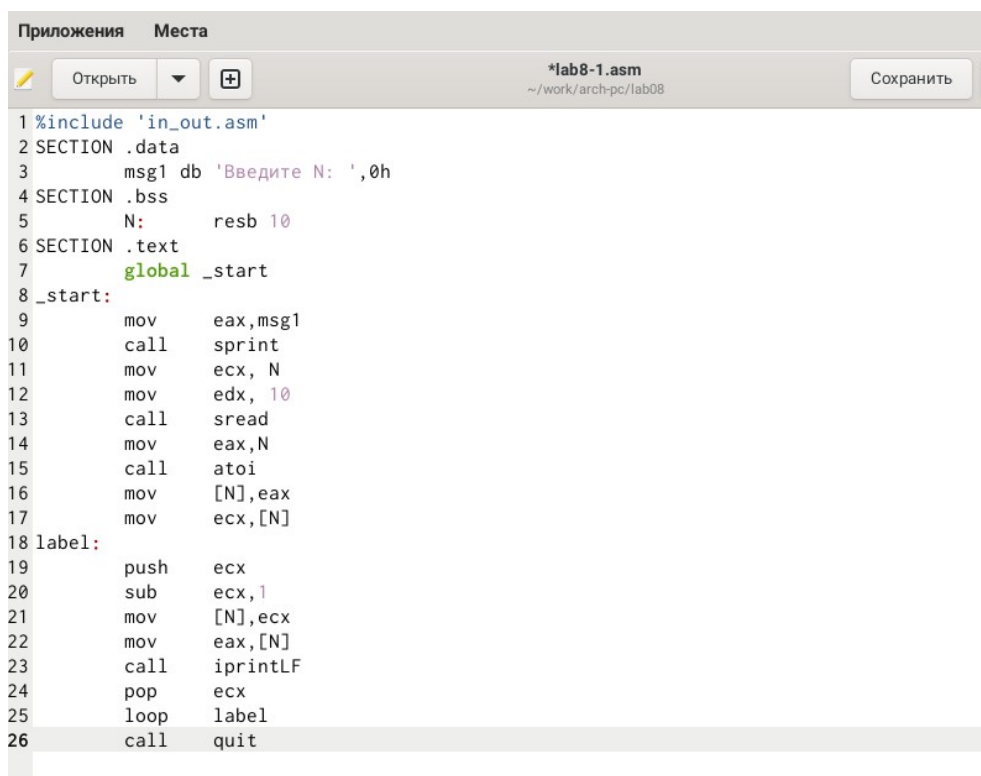
```

kstanchila@dk3n55 ~/work/arch-pc/lab08 $ nasm -f elf lab8-1.asm
lab8-1.asm:9: error: symbol 'msg1' not defined
kstanchila@dk3n55 ~/work/arch-pc/lab08 $ gedit lab8-1.asm
kstanchila@dk3n55 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-1 lab8-1.o
kstanchila@dk3n55 ~/work/arch-pc/lab08 $ ./lab8-1
Введите N: 3
3
2
1
kstanchila@dk3n55 ~/work/arch-pc/lab08 $

```

Рис. 4.1.3 — Вывод программы lab8-1.asm

Данный пример показывает, что использование регистра `ecx` в теле цикла `loop` может привести к некорректной работе программы. Изменила текст программы добавив изменение значение регистра `ecx` в цикле. Создала исполняемый файл и проверила его работу. Регистр `ecx` принимает значения равные $N - 1$, `ecx` — 1 в цикле. Число проходов цикла не соответствует значению N , введенному с клавиатуры. Для использования регистра `ecx` в цикле и сохранения корректности работы программы можно использовать стек. Внесла изменения в текст программы, добавив команды `push` и `pop` (добавления в стек и извлечения из стека) для сохранения значения счетчика цикла `loop` (Рис. 4.1.4).



```

Приложения  Места
*lab8-1.asm
~/work/arch-pc/lab08
Сохранить

1 %include 'in_out.asm'
2 SECTION .data
3     msg1 db 'Введите N: ',0h
4 SECTION .bss
5     N:    resb 10
6 SECTION .text
7     global _start
8 _start:
9     mov     eax,msg1
10    call    sprint
11    mov     ecx, N
12    mov     edx, 10
13    call    sread
14    mov     eax,N
15    call    atoi
16    mov     [N],eax
17    mov     ecx,[N]
18 label:
19    push    ecx
20    sub     ecx,1
21    mov     [N],ecx
22    mov     eax,[N]
23    call    iprintLF
24    pop     ecx
25    loop    label
26    call    quit

```

Рис. 4.1.4 — Текст программы lab8-1.asm

Создала исполняемый файл и проверила его работу. Число проходов цикла значению N соответствует введенному с клавиатуры (Рис. 4.1.5).



```
kstanchila@dk6n17 - lab08
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit lab8-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ nasm -f elf lab8-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-1 lab8-1.o
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./lab8-1
Введите N: 3
2
1
0
```

Рис. 4.1.5 — Вывод программы lab8-1.asm

4.2 Обработка аргументов командной строки

2. При разработке программ иногда встает необходимость указывать аргументы, которые будут использоваться в программе, непосредственно из командной строки при запуске программы. При запуске программы в NASM аргументы командной строки загружаются в стек в обратном порядке, кроме того в стек записывается имя программы и общее количество аргументов. Последние два элемента стека для программы, скомпилированной NASM, – это всегда имя программы и количество переданных аргументов. Таким образом, для того чтобы использовать аргументы в программе, их просто нужно извлечь из стека. Обработку аргументов нужно проводить в цикле. Т.е. сначала нужно извлечь из стека количество аргументов, а затем циклично для каждого аргумента выполнить логику программы. В качестве примера рассмотрим программу, которая выводит на экран аргументы командной строки. Внимательно изучила текст программы Листинга 8.2 (Рис. 4.2.1).


```

#include    'in_out.asm'

SECTION .text
global _start

_start:
    pop     ecx          ; Извлекаем из стека в `ecx` количество
                        ; аргументов (первое значение в стеке)
    pop     edx          ; Извлекаем из стека в `edx` имя программы
                        ; (второе значение в стеке)
    sub     ecx, 1       ; Уменьшаем `ecx` на 1 (количество
                        ; аргументов без названия программы)

next:
    cmp     ecx, 0       ; проверяем, есть ли еще аргументы
    jz      _end         ; если аргументов нет выходим из цикла
                        ; (переход на метку `_end`)
    pop     eax          ; иначе извлекаем аргумент из стека
    call    sprintfLF    ; вызываем функцию печати
    loop    next         ; переход к обработке следующего
                        ; аргумента (переход на метку `next`)

_end:
    call    quit

```

Рис. 4.2.1 — Текст листинга 8.2

Создала файл lab8-2.asm в каталоге ~/work/arch-pc/lab08 и ввела в него текст программы из листинга 8.2. Создала исполняемый файл и запустила его, указав аргументы (Рис. 4.2.2).



```

kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit lab8-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ nasm -f elf lab8-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-2 lab8-2.o
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./lab8-2
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./lab8-2 аргумент 1 аргумент 2 'аргумент 3'
аргумент
1
аргумент
2
аргумент 3

```

Рис. 4.2.2 — Вывод программы lab8-2.asm

Количество обработанных аргументов равно исходному количеству элементов

в стеке минус 2.

Рассмотрели еще один пример программы, которая выводит сумму чисел, которые передаются в программу как аргументы. Создала файл lab8-3.asm в каталоге ~/work/archpc/lab08 и ввела в него текст программы из листинга 8.3 (Рис. 4.2.3)..

```
%include          'in_out.asm'

SECTION .data
msg  db "Результат: ",0

SECTION .text
global _start

_start:

    pop  ecx      ; Извлекаем из стека в `ecx` количество
                  ; аргументов (первое значение в стеке)
    pop  edx      ; Извлекаем из стека в `edx` имя программы
                  ; (второе значение в стеке)
    sub  ecx,1    ; Уменьшаем `ecx` на 1 (количество
                  ; аргументов без названия программы)
    mov  esi, 0   ; Используем `esi` для хранения
                  ; промежуточных сумм

next:
    cmp  ecx,0h   ; проверяем, есть ли еще аргументы
    jz   _end     ; если аргументов нет выходим из цикла
                  ; (переход на метку `_end`)
    pop  eax      ; иначе извлекаем следующий аргумент из стека
    call atoi     ; преобразуем символ в число
    add  esi,eax   ; добавляем к промежуточной сумме
                  ; след. аргумент `esi=esi+eax`
    loop next     ; переход к обработке следующего аргумента

_end:
    mov  eax, msg  ; вывод сообщения "Результат: "
    call sprint
    mov  eax, esi  ; записываем сумму в регистр `eax`
    call iprintLF  ; печать результата
    call quit     ; завершение программы
```

Рис. 4.2.3 — Текст листинга 8.3

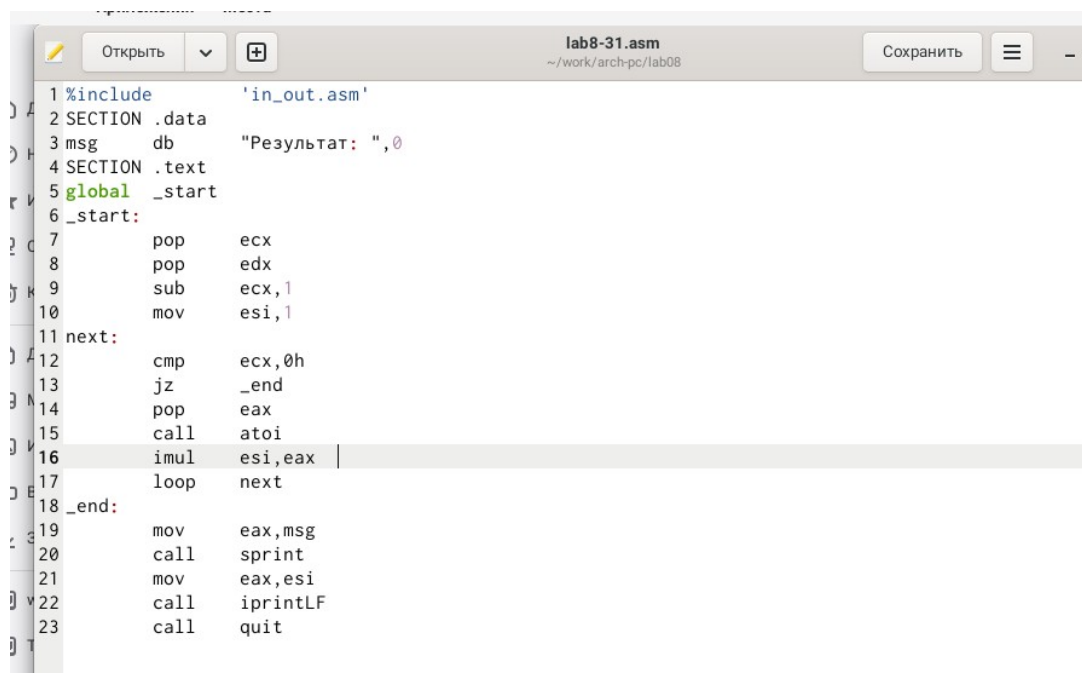
Создала исполняемый файл и запустила его, указав аргументы. Пример

результата работы программы показан на Рис. 4.2.4.

```
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ touch lab8-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit lab8-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ nasm -f elf lab8-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-3 lab8-3.o
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./lab8-3 12 13 7 10 5
Результат: 47
```

Рис. 4.2.4 — Вывод программы lab8-3.asm

Изменила текст программы из листинга 8.3 для вычисления произведения аргументов командной строки (Рис. 4.2.5). Результат работы программы представлен на Рис. 4.2.6.



```
1 %include 'in_out.asm'
2 SECTION .data
3 msg db "Результат: ",0
4 SECTION .text
5 global _start
6 _start:
7     pop     ecx
8     pop     edx
9     sub     ecx,1
10    mov     esi,1
11 next:
12    cmp     ecx,0h
13    jz      _end
14    pop     eax
15    call    atoi
16    imul    esi,eax
17    loop    next
18 _end:
19    mov     eax,msg
20    call    sprintf
21    mov     eax,esi
22    call    iprintLF
23    call    quit
```

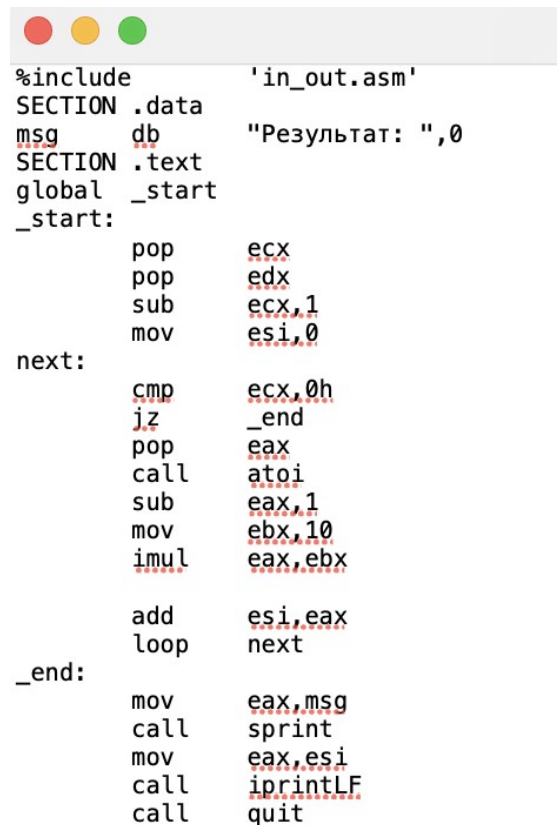
Рис. 4.2.5 — Исправленный текст программы lab8-3.asm

```
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit lab8-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ touch lab8-31.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit lab8-31.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ nasm -f elf lab8-31.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o lab8-31 lab8-31.o
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./lab8-31 2 3 6
Результат: 36
kstanchila@dk6n17 ~/work/arch-pc/lab08 $
```

Рис. 4.2.6 — Вывод программы lab8-3.asm

4.3 Задание для самостоятельной работы

1. Написала программу, которая находит сумму значений функции $f(x)$ для $x = x_1, x_2, \dots, x_n$, т.е. программа должна выводить значение $f(x_1) + f(x_2) + \dots + f(x_n)$. Значения x_i передаются как аргументы. Вид функции: $10(x - 1)$. Создала исполняемый файл и проверила его работу на нескольких наборах $x = x_1, x_2, \dots, x_n$. (Рис. 4.3.1, Рис. 4.3.2).



```
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
    pop    ecx
    pop    edx
    sub    ecx,1
    mov    esi,0
next:
    cmp    ecx,0h
    jz     _end
    pop    eax
    call   atoi
    sub    eax,1
    mov    ebx,10
    imul   eax,ebx
    add    esi,eax
    loop   next
_end:
    mov    eax,msg
    call   sprintf
    mov    eax,esi
    call   iprintLF
    call   quit
```

Рис. 4.3.1 — Текст самостоятельного задания №1



```
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ touch hw1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit hw1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ nasm -f elf hw1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o hw1 hw1.o
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./hw1 3
Результат: 20
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./hw1 2
Результат: 10
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./hw1 4
Результат: 30
kstanchila@dk6n17 ~/work/arch-pc/lab08 $
```

Рис. 4.3.2 — Результат выполнения самостоятельного задания №1

5 Выводы

В результате выполнения данной лабораторной работы я приобрела навыки написания программ с использованием циклов, а также научилась обрабатывать аргументы командной строки.

Список литературы

1. Курс на ТУИС
2. Лабораторная работа №7
3. Программирование на языке ассемблера NASM Столяров А. В